OpenStackとAOSPプロジェクトを対象に
逆強化学習を用いたモジュール別レビュー動態予測

和歌山大学
◯ 橋本一輝，伊原彰紀

wakayama
univ.

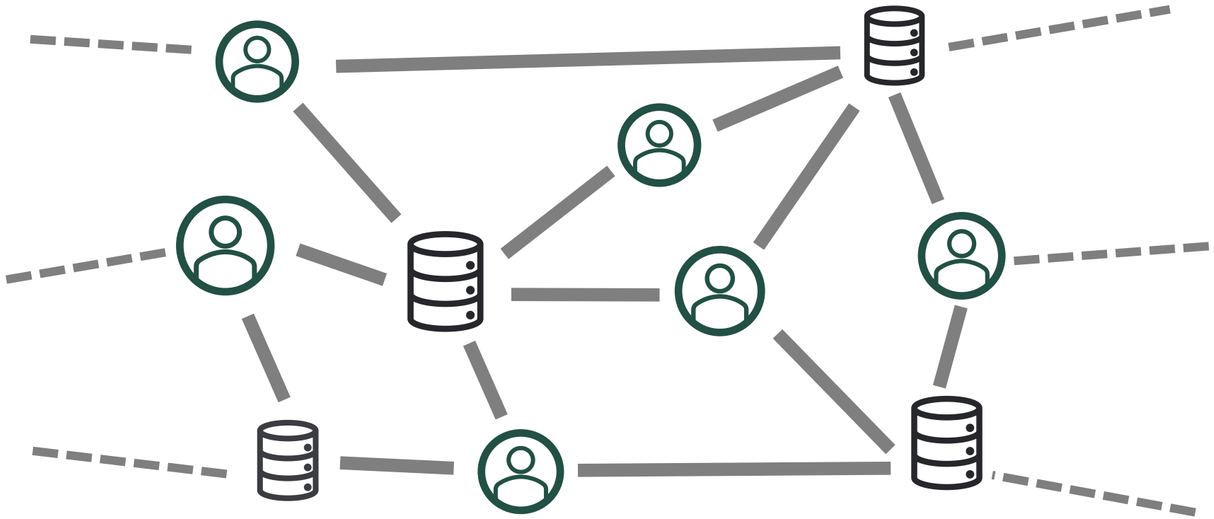
背景：オープンソースソフトウェア（OSS）開発の形態

地理的に分散した開発者が自主的に参加するバザール方式[1]

- 多様なボランティア開発者の貢献により成り立つ

開発者の多様な貢献の動機

- 知的探究心，技術向上，実績など貢献の動機が開発者により異なる[2]



[1] Raymond, E.: The cathedral and the bazaar, Knowledge, Technology & Policy,
[2] Toward an understanding of the motivation of open source software developers, 25th International Conference on Software Engineering, 2003. Proceedings.

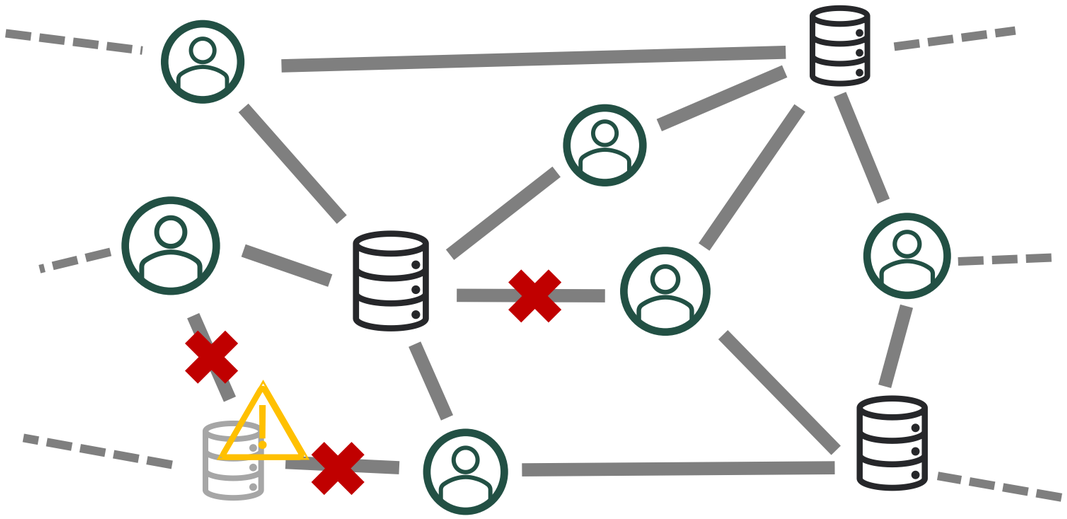
プロジェクトの持続可能性について

TODO:引用

課題

- 貢献するタイミングや量が見積りにくい
- 開発者の流動性が高い
  - 多くの開発者は一度の貢献で離脱[3]

開発者の離脱により保守運用ができなくなる



従来研究

要因分析

開発者の継続・離脱要因について

個人要因と組織要因が影響

予測

継続予測

- 参加後 1 か月の累積活動量から、1 年後以降も貢献するか予測
- 開発者が1年後などに同じプロジェクトへ貢献するか予測

タスク予測

ある時点のレビュー依頼に対して、開発者が貢献するか否かを予測

[3] Lee, A. and Carver, J. C.: Are One-Time Contributors Different? A Comparison to Core and Periphery Developers in FLOSS Repositories, 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)

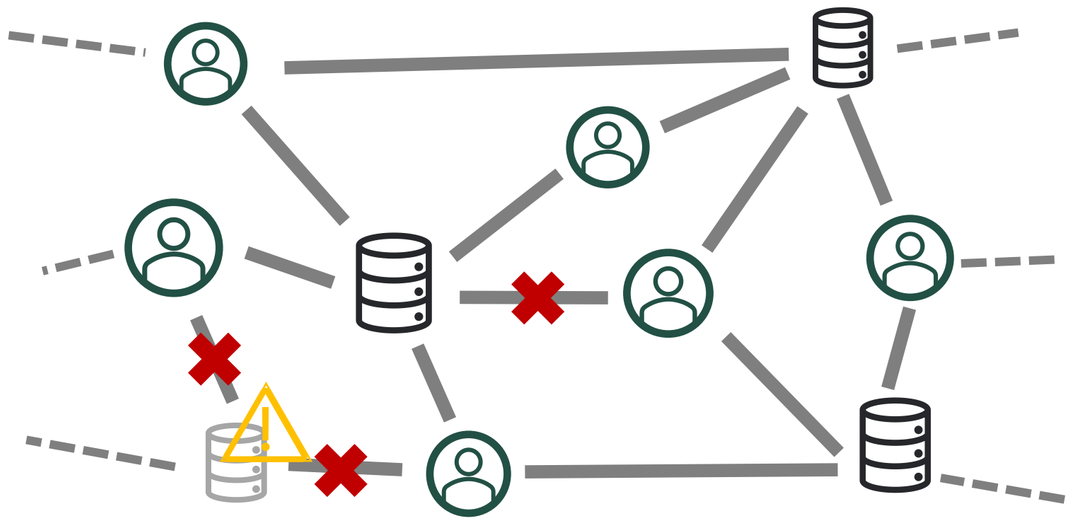
プロジェクトの持続可能性について

TODO:引用

課題

- 貢献するタイミングや量が見積りにくい
- 開発者の流動性が高い
  - 多くの開発者は一度の貢献で離脱[3]

開発者の離脱により保守運用ができなくなる



従来研究

要因分析

開発者の継続・離脱要因について

個人要因と組織要因が影響

予測

継続予測

- 参加後 1 か月の累積活動量から、1 年後以降も貢献するか予測
- 開発者が1年後などに同じプロジェクトへ貢献するか予測

タスク予測

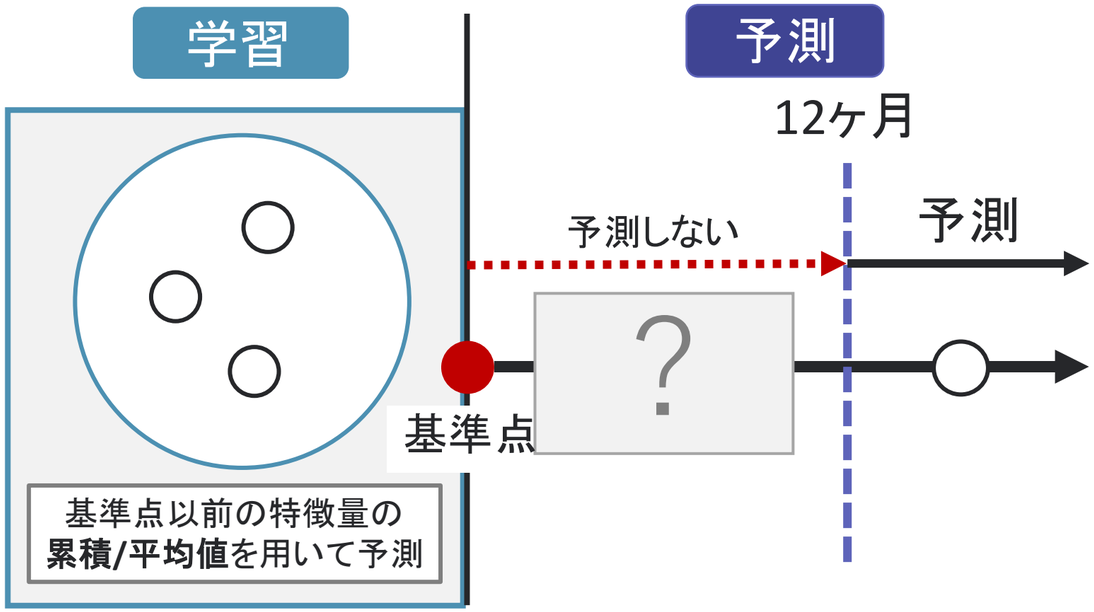
ある時点のレビュー依頼に対して、開発者が貢献するか否かを予測

[3] Lee, A. and Carver, J. C.: Are One-Time Contributors Different? A Comparison to Core and Periphery Developers in FLOSS Repositories, 2017 ACM/IEEE International Symposium on Empirical Software Engineering and Measurement (ESEM)

従来研究と本研究の予測する期間の違い

従来研究

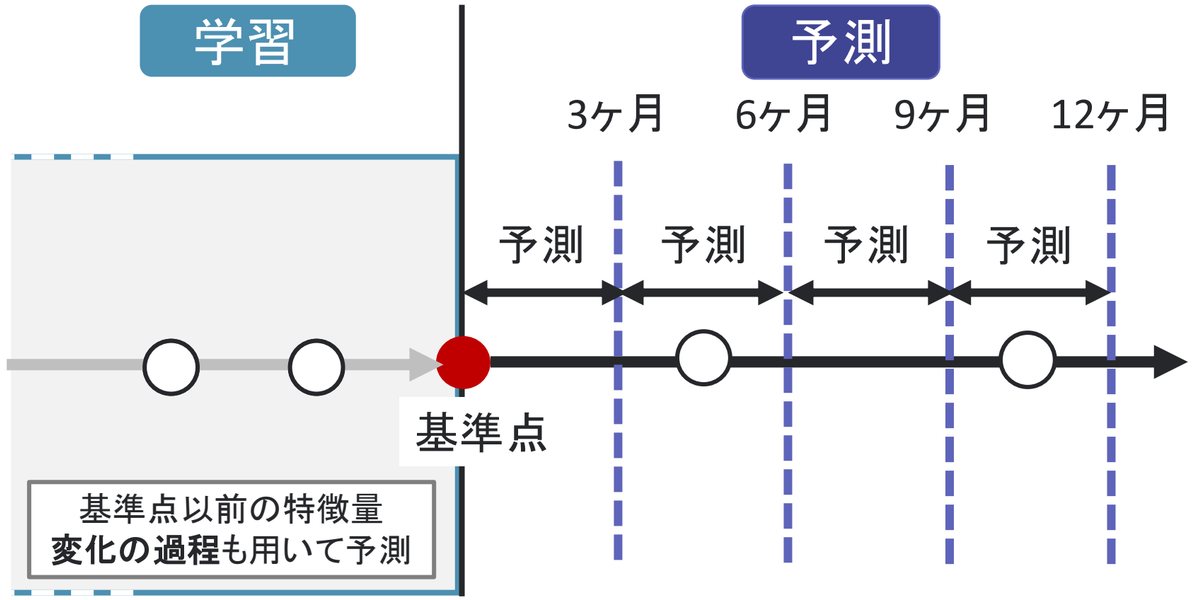
- 特定期間以降の貢献の有無を予測



開発者の一時的な離脱を予測できない

本研究

- 将来の貢献を区間で分割し予測



開発者の離脱・復帰などの時系列パターンも予測

従来研究との予測する対象の粒度の違い

従来研究

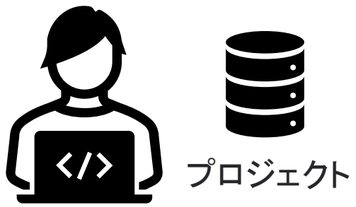
- 特定期間以降の貢献の有無を予測

開発者単位

この開発者は将来も活動しているか？

開発者が残っていても，そのモジュールを
レビューする人がいなければ，保守は困難

novaプロジェクトに貢献するか



本研究

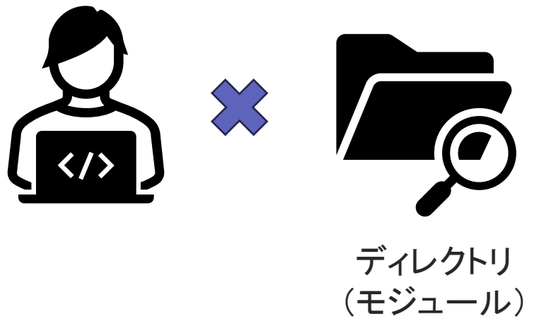
- 将来の貢献を区間で分割し予測

開発者 × ディレクトリ

このディレクトリに，将来この開発者は貢献するか？

モジュール単位で
貢献者を確保できるかを見積もることが可能

novaプロジェクトのnova/computeに貢献するか



例:
- nova/compute
- nova/API
- neutron/agent

目的・キーアイデア

目的

- モジュール単位での，開発者の離脱・復帰などの時系列パターンを考慮した予測

キーアイデア

- 開発者を「レビュー依頼に対して 承諾／拒否 を選ぶ主体」として捉える逆強化学習

累積統計量



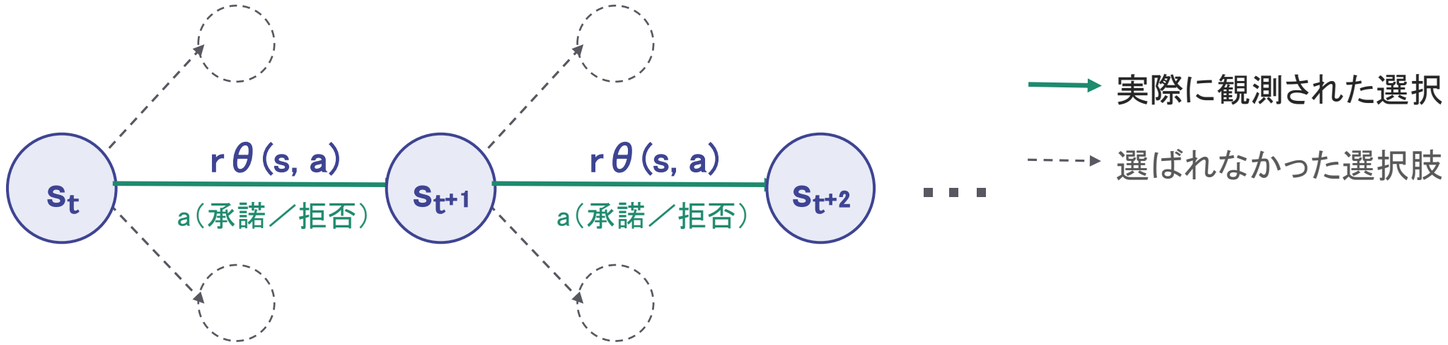
月次の承諾（1）／非承諾（0）



承諾率 = 0.50

いつ止まったかは平均に埋もれる

状態遷移として学習（IRL）



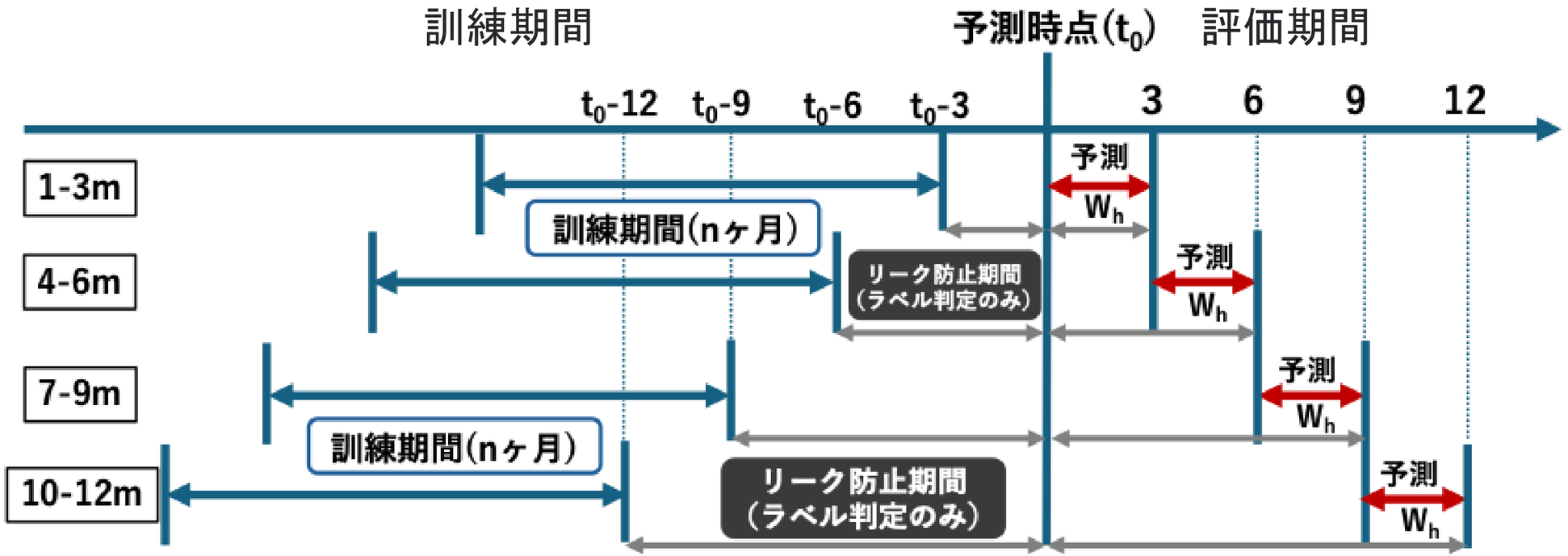
各時点で他の選択肢もあった中で、実際にどれを選んだかを軌跡として学習する

他の選択肢もあった中である選択が繰り返された ⇒ 判断基準 $r_\theta(s, a)$ を逆算できる

アプローチ
モデルの全体像

開発者×ディレクトリのペアで各期間の貢献を予測する4つのモデルを作成

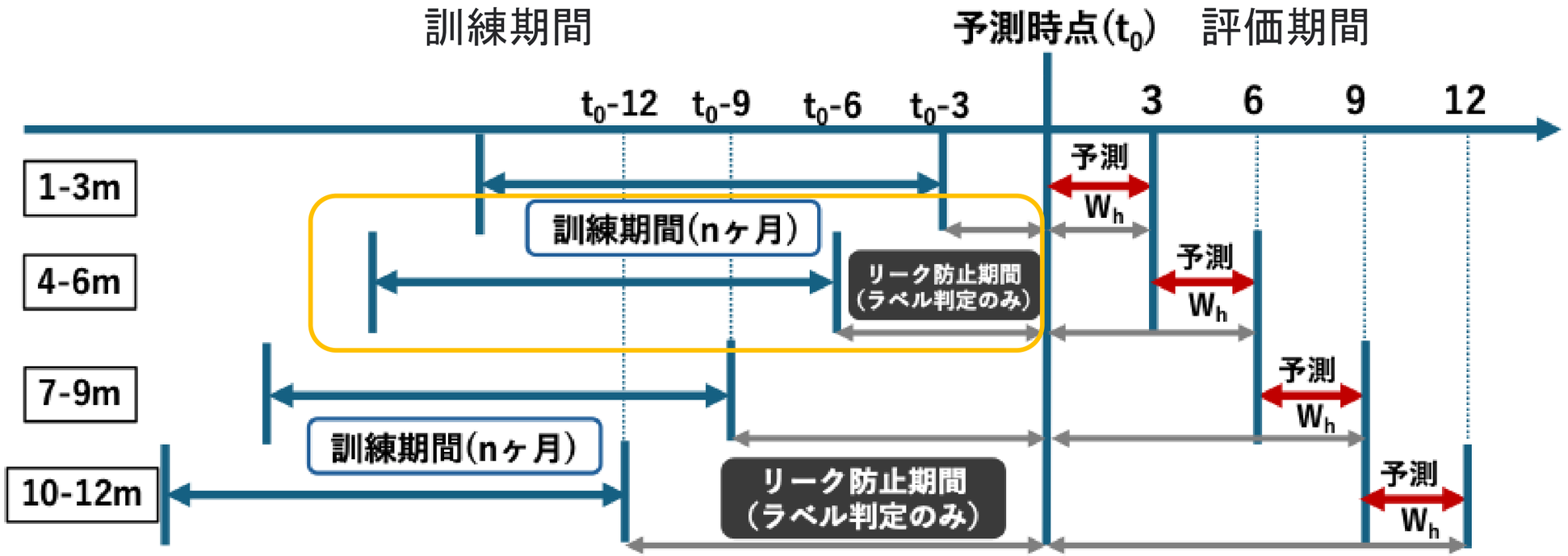
- 予測時点(t0)以降の期間を3ヶ月ずつ(W_1:1−3m/W_2:4−6m/W_3:7−9m/W_4:10−12m)予測



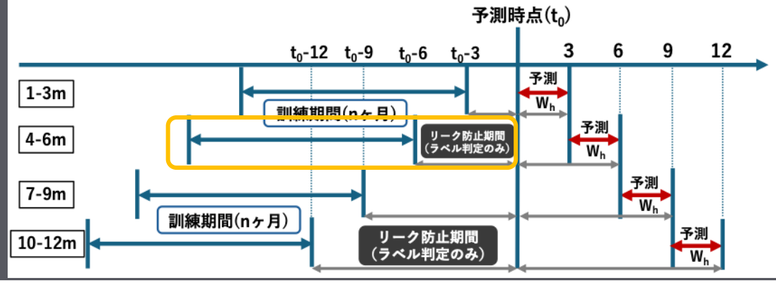
アプローチ
モデルの全体像

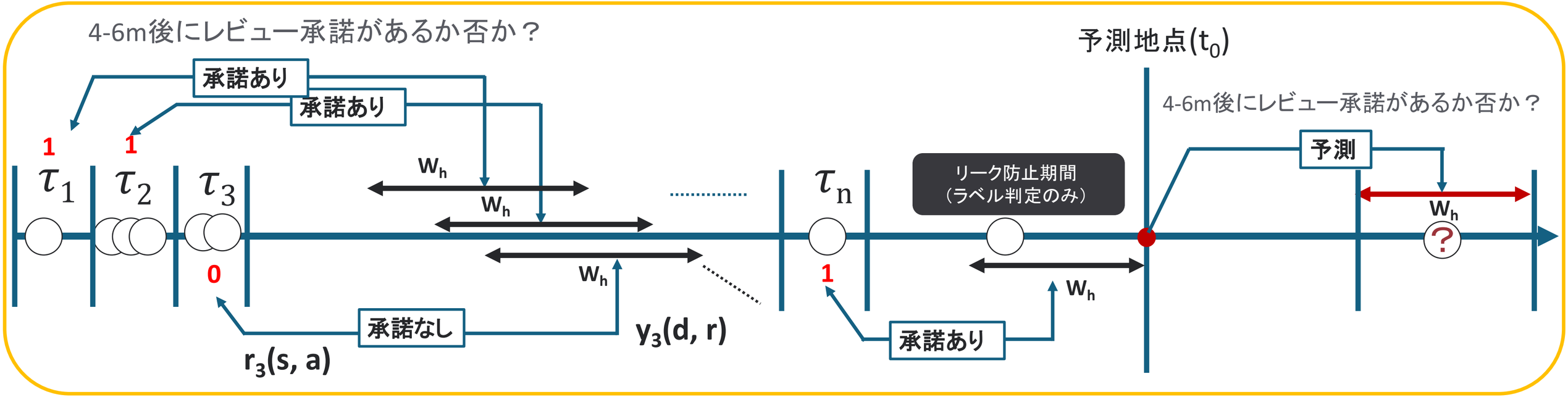
開発者×ディレクトリのペアで各期間の貢献を予測する4つのモデルを作成

- 予測時点(t0)以降の期間を3ヶ月ずつ(W_1:1−3m/W_2:4−6m/W_3:7−9m/W_4:10−12m)予測



アプローチ
学習方法・予測について（4-6mの例）





■ 状態 s_t：
開発者のそれまでのレビュー履歴
(開発者状態，ディレクトリ親和度，活動特徴の3カテゴリ)

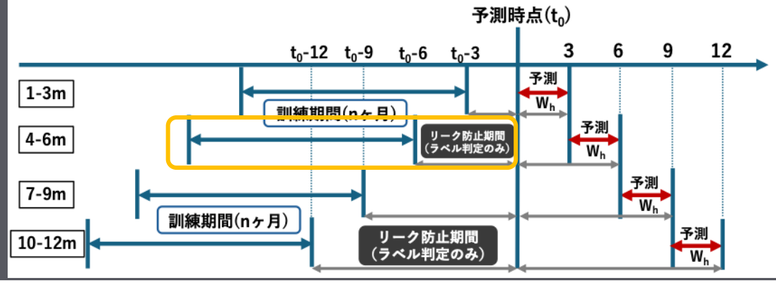
■ 軌跡 τ_t：
状態+ラベルの月次記録。$T = (\tau_1, \ldots, \tau_n)$ として学習

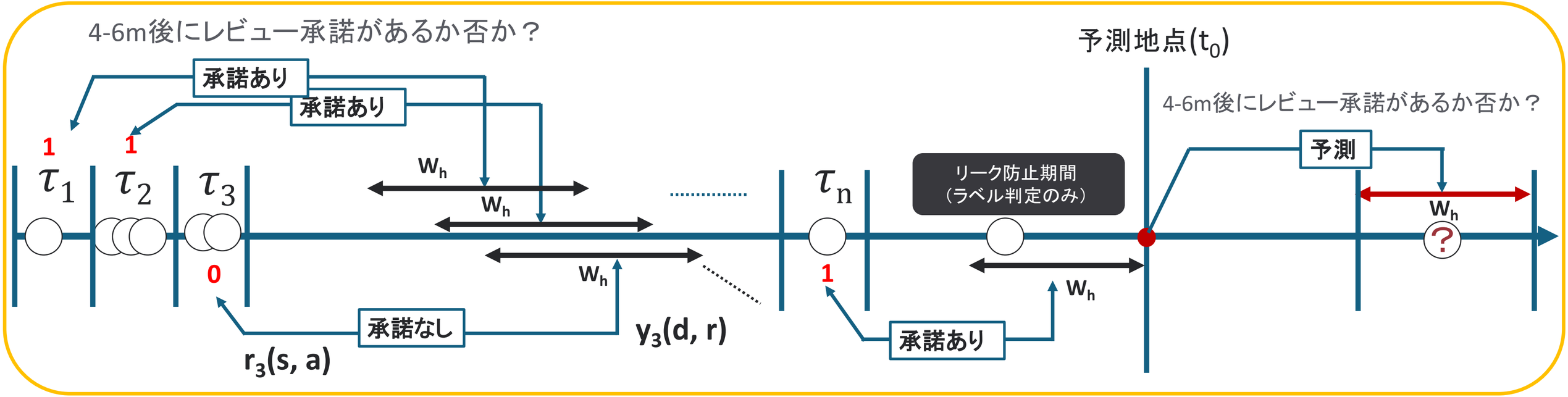
■ 報酬 $r_t(s, a)$：
依頼を受けるかの判断基準．IRLの推定対象

ラベル$y_t(d, r)$：各月τ_tにW_h内の貢献で付与

+1	依頼あり，貢献あり ＝ 正例（重み 1.0）
0	依頼あり，貢献なし ＝ 強い負例（重み 1.0）
0*	依頼なし ＝ 弱負例（重み 0.5）

アプローチ
学習方法・予測について（4-6mの例）





■ 状態 s_t：
開発者のそれまでのレビュー履歴
(開発者状態，ディレクトリ親和度，活動特徴の3カテゴリ)

■ 軌跡 τ_t：
状態+ラベルの月次記録。$T = (\tau_1, \ldots, \tau_n)$ として学習

■

ラベル$y_t(d, r)$：各月τ_tにW_h内の貢献で付与

+1	依頼あり，貢献あり ＝ 正例（重み 1.0）
0	依頼あり，貢献なし ＝ 強い負例（重み 1.0）

累積統計量では平均化される「活動の停止と再開」，「関与するディレクトリの移り変わり」等を時間経過順に学習可能

評価方法と比較手法

評価方法

- AUC-ROC: 貢献する開発者としない開発者をどれだけ正しく識別できるか
- 有意性の判定：bootstrap法による95%信頼区間

比較手法：Random Forest

- 従来手法で広く用いられる．決定木の多数決により貢献確率を算出
  - 学習設定は既定値：木の数100，深さ無制限，葉あたり最小サンプル1，乱数シード777，class_weight=balanced（探索なし）

アブレーション：LSTM

- 「時系列を学習したこと」，「報酬関数として定式化したこと」のどちらが有効であったのかを検証するために比較

ケーススタディ

対象プロジェクトの選定

- OpenStack:
  - 公式サービスリポジトリ33件のうち，レビュー履歴のないsunbeamを除く32件のリポジトリ
- AOSP:
  - 活動量に基づきOpenStackと同様に32リポジトリを選定

訓練・評価期間

- 訓練期間3年，評価期間1年，t_0(予測開始地点)2023年1月１日

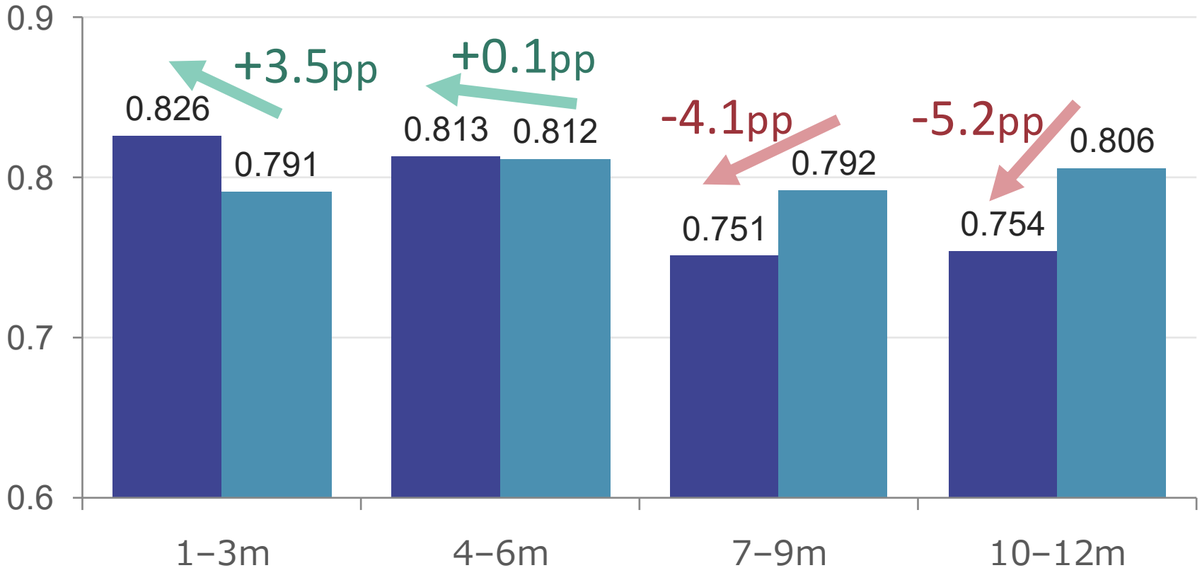
	データセット全体				予測対象（訓練・評価の両期間で観測）	
プロジェクト	リポジトリ	チケット	レビュー依頼行	開発者	開発者	ペア
OpenStack	32	42,441	161,111	2,372	**436**	**4,534**
AOSP	32	408,574	574,810	8,901	**2,508**	**26,830**

結果

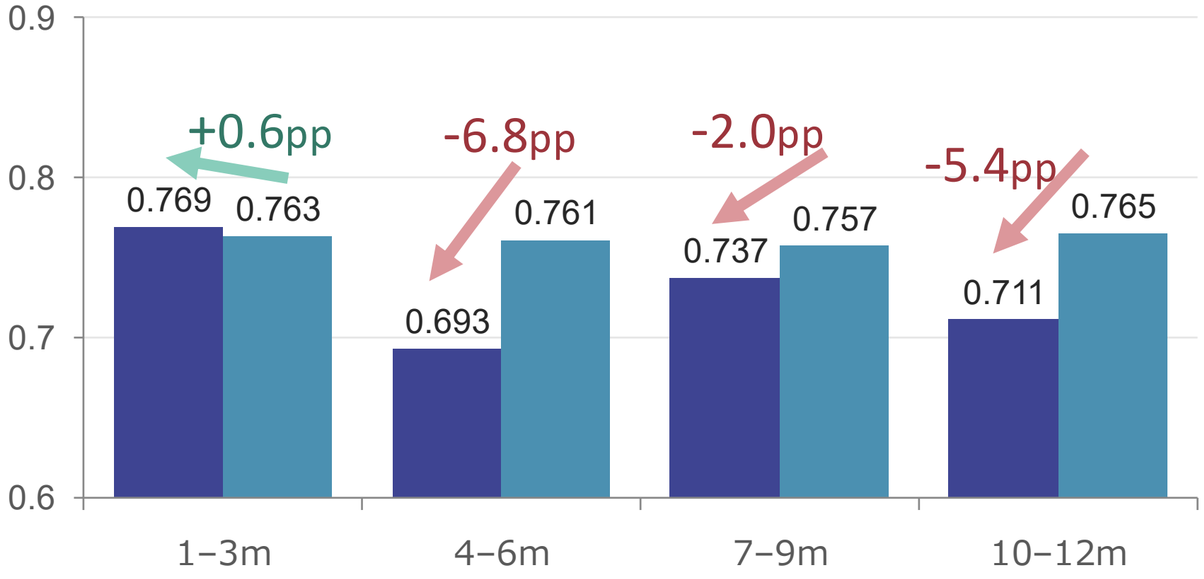
IRLとRFは同程度の予測精度

- 4期間のAUCではOpenStack, AOSPともにRFが上回った
- 短期(1–3m)ではIRLがわずかに優位な傾向が見られた

OpenStack（ペア単位、n = 4,534）



AOSP（ペア単位、n = 26,830）

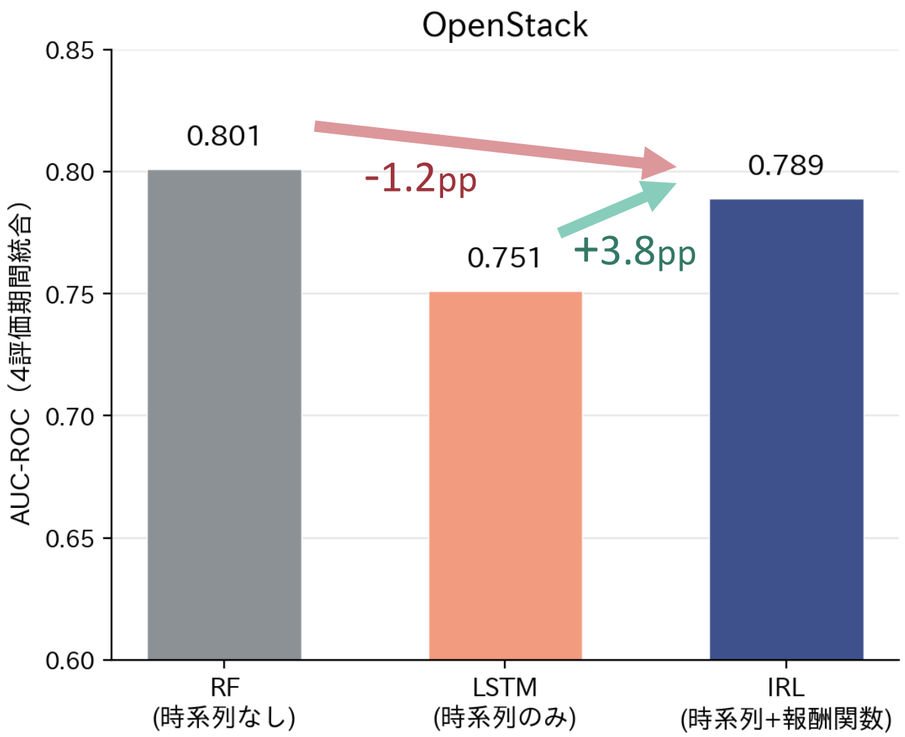


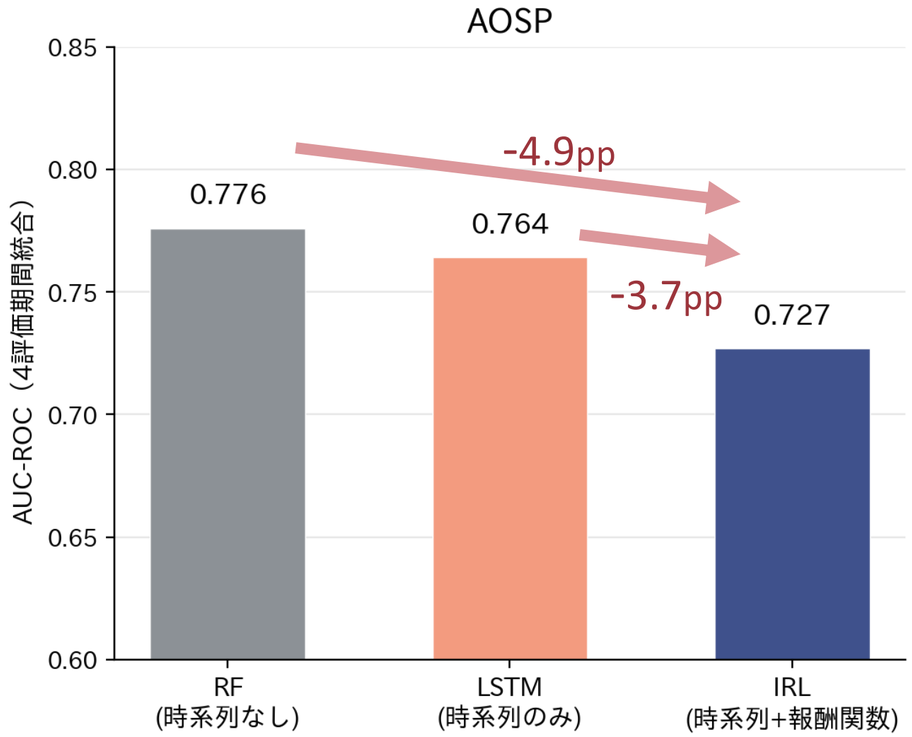
結果：時系列や報酬関数の定式化のアブレーション

グラフにする

時系列モデル単体では不十分・報酬関数としての定式化による補完効果もエコシステムに依存する

4つの評価期間を統合し，IRL, LSTM, RFの3つの精度を比較



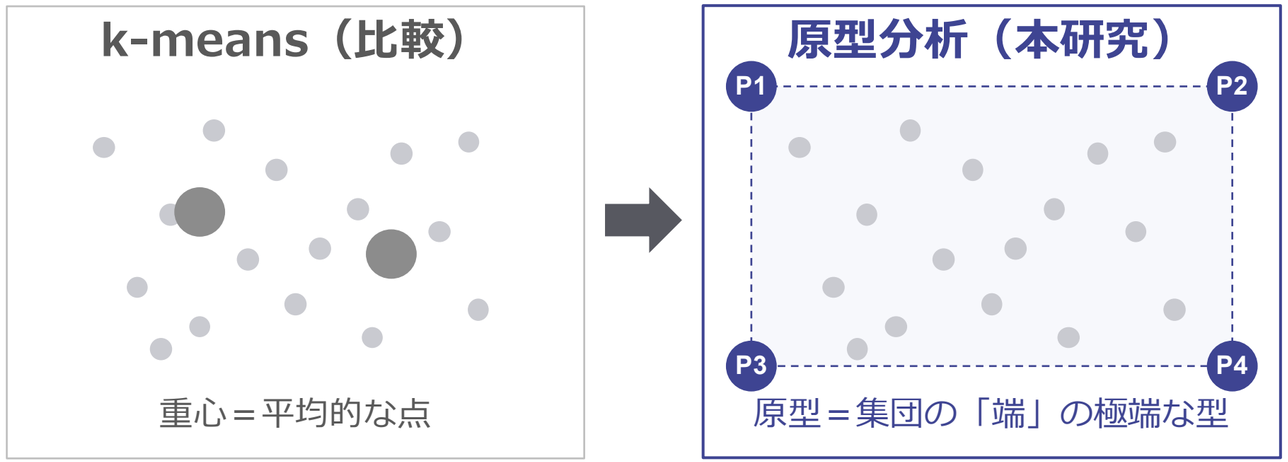


アプローチ：比較手法との補完性の検証

原型分析[1]

極端なパターンを K 個抽出し、各データ点をその重み付き和で近似する手法

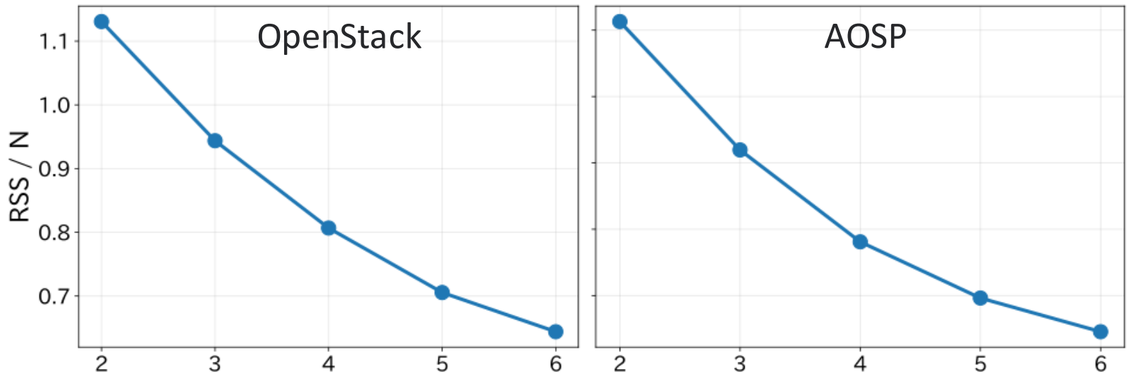
■ K-meansとの違い



中心が平均に近く，クラスタ間での特徴に差が出にくい

原型は特徴量の偏りが大きくタイプごとの特徴を分離しやすい

■ 原型数Kの決め方



図：原型数 K に対する平均残差平方和の推移

- RSS /Nの改善が小さくなる点（エルボー）で決定
- K=3→4の改善が大きく，5以降は小さいため両プロジェクトでK=4

分析対象：t0 直前6か月に履歴がある開発者（OpenStack 289 開発者・3,747 ペア，AOSP 1,248 開発者・21,291 ペア）

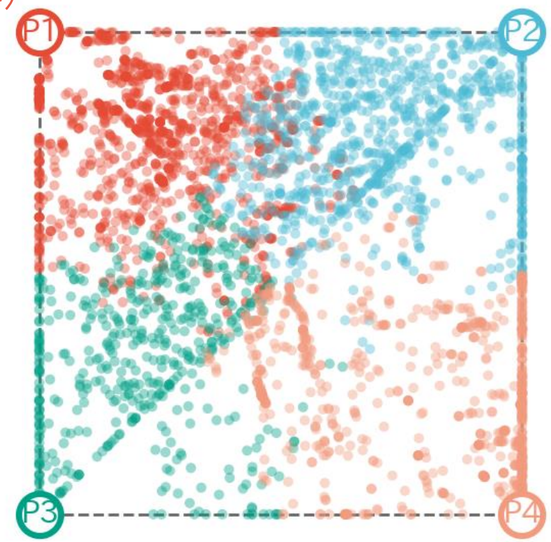
[1] Cutler, A. and Breiman, L.: Archetypal analysis, Technometrics, Vol. 36, No. 4, pp. 338–347 (1994).

OpenStackのケーススタディ

結果：開発者タイプごとに、当たるモデルが異なる（OpenStack・ペア単位）

P1:他プロジェクト
経験者（n=1,284）

P2:関与低下型（n=928）



P3:高承諾型（n=539）

P4:低活動・低承諾型（n=926）

評価期間別 Δ ＝ IRL − RF（pp）

	1–3m	4–6m	7–9m	10–12m	ΣΔ
P1	+9.0	+3.9	+4.2	−5.4	+11.7
P2	+2.0	−1.5	−0.1	−4.2	−3.8
P3	+5.8	−2.3	−7.0	−0.7	−4.2
P4	−0.4	−0.5	−8.7	−5.9	−15.5

P1:他プロジェクト経験者

他プロジェクト貢献は多いが，対象ディレクトリの実績は乏しい

4原型中最多のn(1,284)を持つP1で，4期間中3期間 IRL が優位

対象ディレクトリでの累積統計量が少ないため，
時系列情報を用いるIRLが有効であることが考えられる

P2:関与低下型

対象ディレクトリの最終活動からの経過時間が長く，承諾率は低め

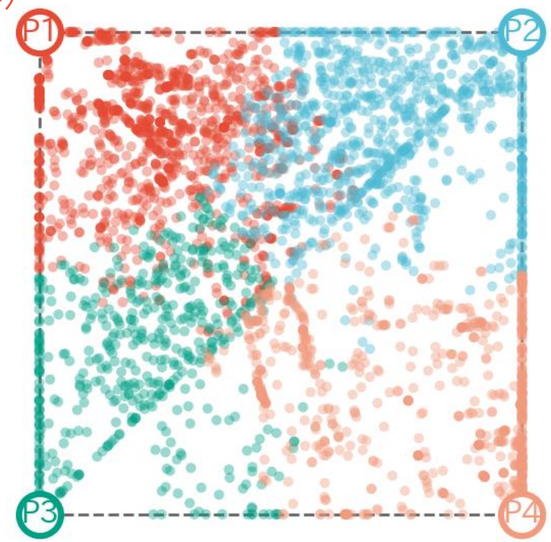
短期(1–3m)のみ IRL 優位，中長期は RF が優位

短期は活動パターンの変化をIRLが捉えたことにより優位になり，
その影響が中長期にかけて弱まりRFが有効になると考えられる

結果：開発者タイプごとに、当たるモデルが異なる（OpenStack・ペア単位）

P1:他プロジェクト
経験者（n=1,284）

P2:関与低下型（n=928）



P3:高承諾型（n=539）

P4:低活動・低承諾型（n=926）

評価期間別 Δ ＝ IRL − RF（pp）

	1–3m	4–6m	7–9m	10–12m	ΣΔ
P1	+9.0	+3.9	+4.2	−5.4	+11.7
P2	+2.0	−1.5	−0.1	−4.2	−3.8
P3	+5.8	−2.3	−7.0	−0.7	**−4.2**
P4	−0.4	−0.5	−8.7	−5.9	**−15.5**

P3:高承諾型

承諾率・活動頻度がともに高い

短期(1–3m)のみ IRL 優位，中長期は RF が優位

承諾率が高く，活動頻度も高い開発者は，
累積統計量(RF)で十分に説明できることが考えられる

P4:低活動・低承諾型

活動頻度・承諾率が低い

全期間RF が優位

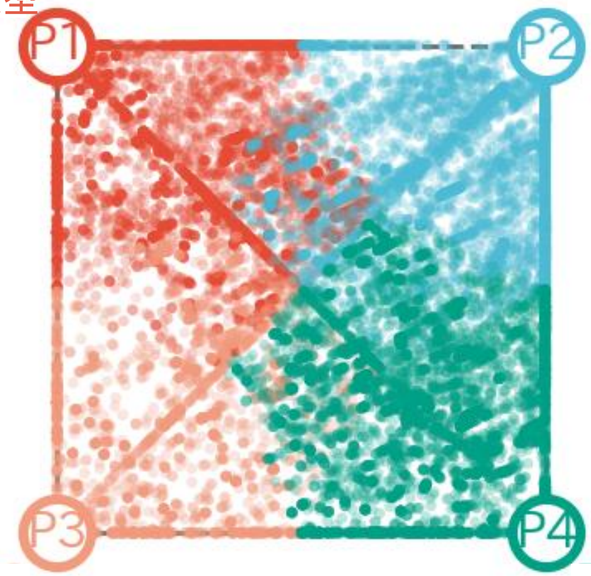
活動が少ないとIRLの学習に十分な行動軌跡が得られないことが考えられる

AOSPのケーススタディ

結果：開発者タイプごとに、当たるモデルが異なる（AOSP・ペア単位）

P1:高活動・
ディレクトリ未経験型

P2:ディレクトリ経験者・高関与型



P3:低承諾型

P4: 高承諾・汎用型

評価期間別 Δ ＝ IRL − RF（pp）

	1–3m	4–6m	7–9m	10–12m	ΣΔ
P1	+0.4	−3.3	−0.2	−8.6	−11.6
P2	−4.4	−11.4	−9.8	−12.4	−38.0
P3	+1.5	−0.5	−2.9	−2.6	−4.6
P4	+4.6	+2.3	+3.9	+3.5	+14.3

P1:高活動・ディレクトリ未経験型

活動頻度は高いが，対象ディレクトリの実績は乏しい

短期(1–3m)のみ IRL 優位，中長期は RF が優位

対象ディレクトリでの累積統計量が少ないため，
時系列情報を用いるIRLが有効であることが考えられる

P2:ディレクトリ経験者・高関与型

対象ディレクトリへの貢献が多く，活動頻度も高い

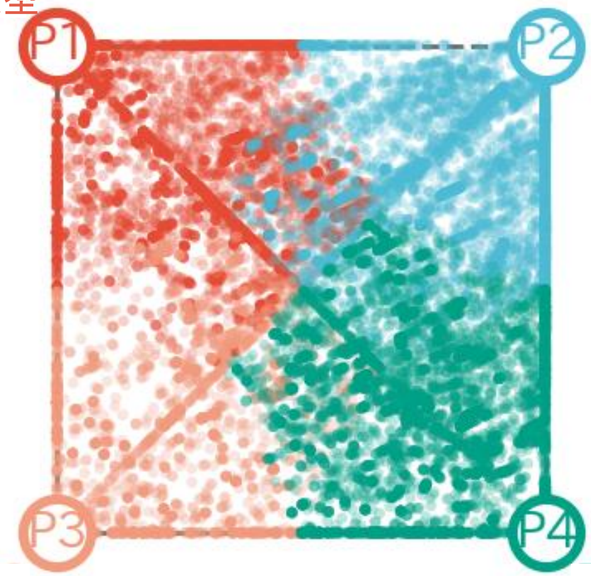
全期間RF が優位

対象ディレクトリへの貢献が多く，活動頻度も高い開発者は，
累積統計量(RF)で十分に説明できることが考えられる

結果：開発者タイプごとに、当たるモデルが異なる（AOSP・ペア単位）

P1:高活動・
ディレクトリ未経験型

P2:ディレクトリ経験者・高関与型



P3:低承諾型

P4: 高承諾・汎用型

評価期間別 Δ ＝ IRL − RF（pp）

	1–3m	4–6m	7–9m	10–12m	ΣΔ
P1	+0.4	−3.3	−0.2	−8.6	−11.6
P2	−4.4	−11.4	−9.8	−12.4	−38.0
P3	+1.5	−0.5	−2.9	−2.6	−4.6
P4	+4.6	+2.3	+3.9	+3.5	+14.3

P3:低承諾型

活動頻度，承諾率が低い

短期(1–3m)のみ IRL 優位，中長期は RF が優位

基本的にはOpenStackの結果と同様に，
活動が少ないとIRLが十分な行動軌跡がなくうまく学習できない

P4: 高承諾・汎用型

承諾率・ディレクトリのハブ度が高い一方で，活動頻度が低い

4原型中最多のn(8,923)を持つP4で，全期間IRLが優位

OpenStackの結果と異なり，
高承諾で活動頻度が低くてもIRLが優位になった

ケーススタディまとめ

原型となるべく対応させてみせ
or
各結果と距離を近くする

軌跡が短い開発者

- IRLは行動パターンをうまく学習できないが，RFは累積の活動量が少ないこと自体が活動停止の予測に寄与する

承諾パターンが変化する開発者

- RFは訓練期間の低い承諾率を重視するが，IRLは行動軌跡から状態の変化を活用することができる

予測期間の長さ

- 長期になるほど，一時的な行動パターンの変化が平均化され，累積特徴量を用いるRFが安定

エコシステムの多様性

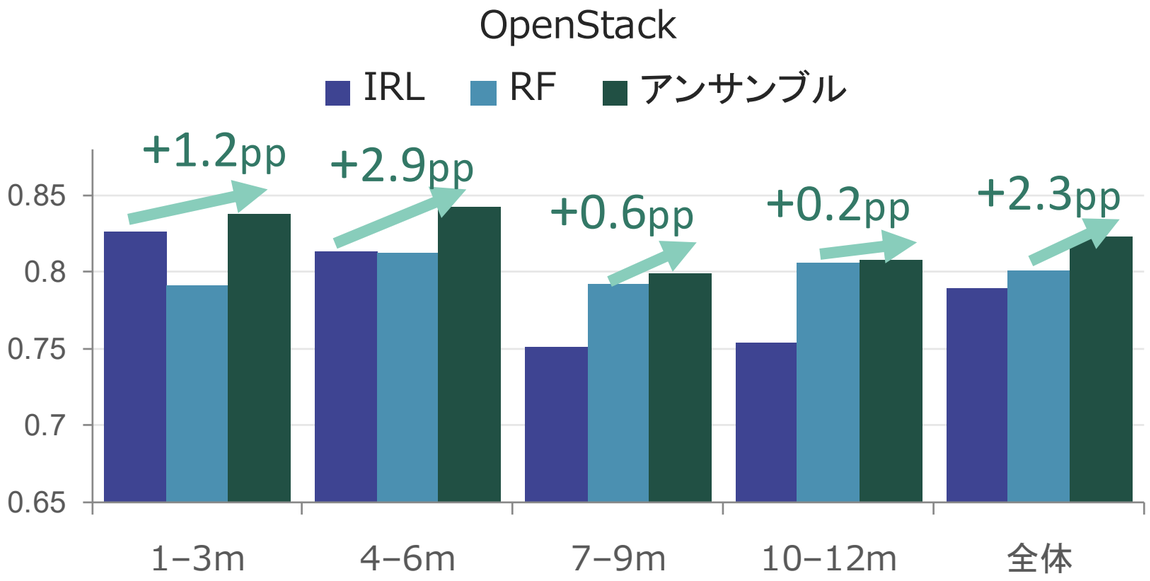
- 規模が大きいほど1つのディレクトリに多様な開発者が関与し，行動軌跡から一貫した判断基準を推定しにくくなる

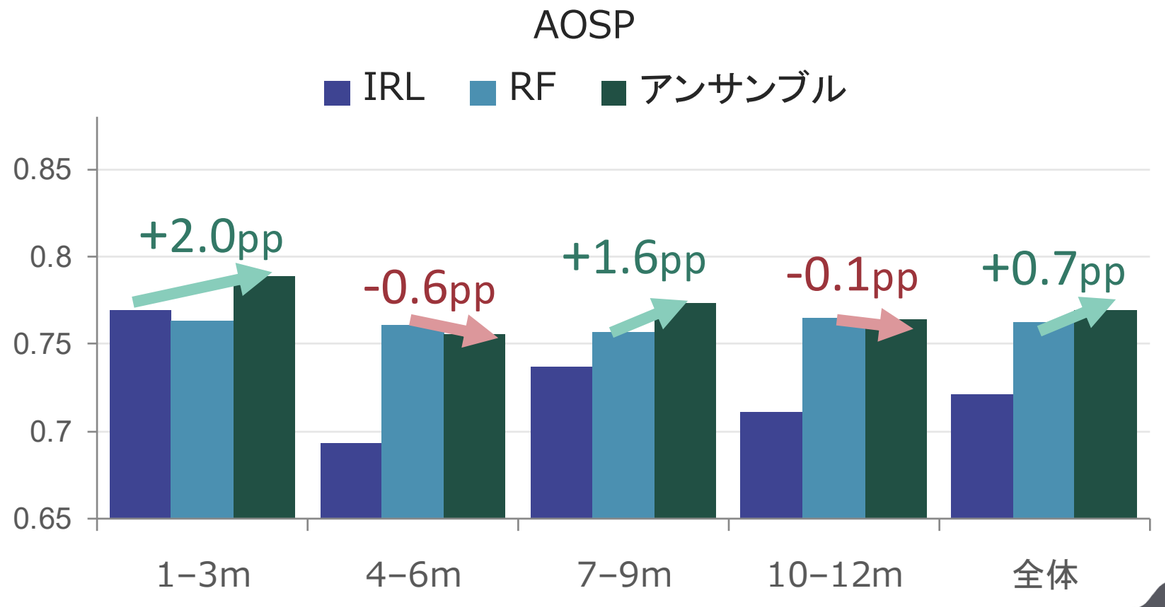
追加実験:アンサンブル

両エコシステムで予測精度改善： OpenStack +2.3 [+1.7, +2.8], AOSP +0.7 [+0.5, +1.0]

IRLとRFが出力する確率を単純平均し予測値として評価

- 互いの誤りを相殺し，精度が向上させることが目的





まとめ・今後の展望

スクショ？

まとめ

- 開発者×ディレクトリのペアに対して，レビュー貢献を連続的に予測するIRLモデルを構築
- RFとIRLを比較したところ予測精度は同程度であった
  - 原型分析により，予測が得意な開発者のタイプが異なる可能性を示し，両モデルの予測確率を単純平均するアンサンブルモデルにより精度が向上した

今後の展望

- 推定した報酬関数を環境のフィードバックとして組み込み，開発者の継続確率を考慮したレビュー依頼の推薦方策の設計・評価
- 原型や評価期間に応じてIRL/RFを選択的に採用することによる精度改善

補足資料

アプローチ（逆強化学習モデルの構築，比較手法との比較）
開発者の状態s_tを表現する特徴量

アプローチかキーアイデにマージ

3つのカテゴリ計37次元の特徴量を用いる

カテゴリ	次元数	特徴量（抜粋）
開発者状態	**26**	累積レビュー数・受諾率・直近性・レビュー承諾／非承諾の変動回数 等
ディレクトリ親和度	**6**	過去レビュー数・直近性・受諾率・所有者重なり 等
活動特徴	**5**	行動強度・変更行数・レビュー規模・応答時間・協働の反復率

- 従来研究では，評価対象プロジェクト内の貢献のみを特徴量とすることが多い
- 本研究では，他のプロジェクトへの活動も計測対象に含めることで，
  「他プロジェクトでの多忙により一時的に不活発」「開発活動そのものを停止」といった，
  プロジェクト内の履歴のみでは区別できない状況の把握を可能にする